

Unit-3

Arrays

Arrays:

- In Python, array is a collection of items stored at contiguous memory locations.
- The idea is to store multiple items of the same type together.
- Unlike [Python lists](#) (can store elements of mixed types), arrays must have all elements of same type. Having only homogeneous elements makes it memory-efficient.
- It requires a Typecode during initialization to define the C-type.

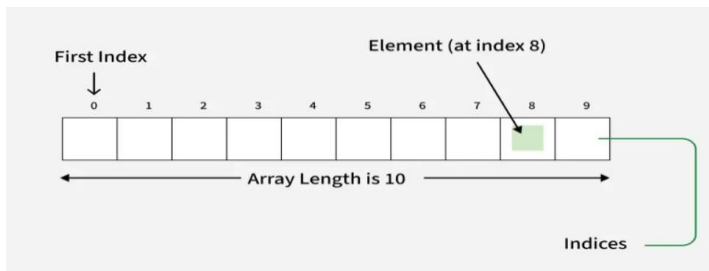
Example:

```
import array as arr
a = arr.array('i', [1, 2, 3])
# accessing First Array
print(a[0])
# adding element to array
a.append(5)
print(a)
```

Creating an Array:

- An array is a collection of elements stored in a single variable. All elements in an array usually have the same data type, and each element can be accessed using an index number.
- In Python, arrays can be created using the array module.
- Import the Array Module
- Before creating an array, you must import the array module.

```
import array
```



Syntax for Creating an Array:

```
array_name = array.array(type_code, [elements])
```

Example:

```
import array
numbers = array.array('i', [10, 20, 30, 40, 50])
print(numbers)
```

Accessing the Elements in an Array:

- Accessing elements means retrieving or getting the values stored in an array. Each element in an array has a position called an index.
- In Python, the index starts from 0.
- In order to access the array items refer to the index number. Use the index operator[] to access an item in a array in Python. The index must be an integer.

Syntax:

```
array_name[index]
```

Example:

```
import array as arr
a = arr.array('i', [1, 2, 3, 4, 5, 6])
print(a[0])
print(a[3])

b = arr.array('d', [2.5, 3.2, 3.3])
print(b[1])
print(b[2])
```

Arrays Bulit-in Methods:

Python has a set of built-in methods that you can use on lists/arrays.

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the first item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

1. append() Method:

- The append() method is used to add an element at the end of the array.

Syntax:

```
array_name.append(value)
```

Example:

```
import array
arr = array.array('i', [10, 20, 30])
arr.append(40)
print(arr)
```

output:

```
array('i', [10, 20, 30, 40])
```

2. insert() Method:

- The insert() method is used to insert an element at a specific position in the array.

Example:

```
import array
arr = array.array('i', [10, 20, 30])
arr.insert(1, 15)
print(arr)
```

3. remove()

- Removes the first occurrence of a value.

Syntax:

```
list.remove(value)
```

Example:

```
a = [10, 20, 30, 20]
a.remove(20)
print(a)
```

4. pop()

- Removes an element using its index position.

Syntax:

```
list.pop(index)
```

Example:

```
a = [1, 2, 3, 4]
```

```
a.pop(2)
```

```
print(a)
```

Python Lists:

- In Python, a list is a built-in data structure that can hold an ordered collection of items.
- Unlike arrays in some languages, Python lists are very flexible
- Can contain duplicate items
- Mutable: items can be modified, replaced, or removed
- Ordered: maintains the order in which items are added
- Index-based: items are accessed using their position (starting from 0)
- Can store mixed data types (integers, strings, booleans, even other lists)

Creating a List:

- Lists can be created in several ways, such as using square brackets, the list() constructor or by repeating elements.

Let's look at each method one by one with example:

1. Using Square Brackets:

- We use square brackets [] to create a list directly.

Example:

```
a = [1, 2, 3, 4, 5] # List of integers  
  
b = ['apple', 'banana', 'cherry'] # List of strings  
  
c = [1, 'hello', 3.14, True] # Mixed data types  
  
print(a)  
  
print(b)  
  
print(c)
```

Output

```
[1, 2, 3, 4, 5]  
['apple', 'banana', 'cherry']  
[1, 'hello', 3.14, True]
```

2. Using list() Constructor:

- We can also create a list by passing an iterable (like a [tuple](#), [string](#) or another list) to the [list\(\)](#) function.

Example:

```
a = list((1, 2, 3, 'apple', 4.5))  
  
print(a)  
  
b = list("GFG")  
  
print(b)
```

Output

```
[1, 2, 3, 'apple', 4.5]  
['G', 'F', 'G']
```

3. Creating List with Repeated Elements:

- We can use the multiplication operator * to create a list with repeated items.

Example:

```
a = [2] * 5
```

```
b = [0] * 7
```

```
print(a)
```

```
print(b)
```

Output:

```
[2, 2, 2, 2, 2]
```

```
[0, 0, 0, 0, 0, 0, 0]
```

List operations:

1. Accessing Elements:

- Elements in a list can be accessed using the index number. Indexing starts from 0.

Example

```
a = [10, 20, 30, 40]
```

```
print(a[0])
```

```
print(a[2])
```

Output

```
10
```

```
30
```

2. Updating (Modifying) Elements:

- Lists are mutable, so elements can be changed.

Example:

```
a = [10, 20, 30]
a[1] = 50
print(a)
```

Output

```
[10, 50, 30]
```

3. Adding Elements:

- Elements can be added to a list using methods like `append()` and `insert()`.

Example:

```
a = [1, 2, 3]
a.append(4)
print(a)
```

Output

```
[1, 2, 3, 4]
```

4. Deleting Elements:

- Elements can be removed using `del`, `remove()`, or `pop()`.

Example

```
a = [10, 20, 30]
del a[1]
print(a)
```

Output

```
[10, 30]
```

5. Concatenation:

- Two lists can be combined using the + operator.

Example:

```
a = [1, 2]
b = [3, 4]
c = a + b
print(c)
```

Output:

```
[1, 2, 3, 4]
```

6. Repetition:

- Lists can be repeated using the * operator.

Example:

```
a = [1, 2]
print(a * 3)
```

Output

```
[1, 2, 1, 2, 1, 2]
```

7. Membership Operation:

- Used to check whether an element is present in the list using in and not in.

Example:

```
a = [10, 20, 30]
print(20 in a)
print(50 in a)
```

Output

```
True
False
```

8. Length of List:

- The `len()` function returns the number of elements in the list.

Example:

```
a = [1, 2, 3, 4]
print(len(a))
```

Output: 4

Built-in Functions for Lists:

- These functions are part of Python's general built-in functions and take the list as an argument.
- **`len(list)`** : Returns the total number of elements in the list.
- **`max(list)`** : Returns the item with the maximum value in the list.
- **`min(list)`** : Returns the item with the minimum value in the list.
- **`sum(list, [start])`** : Returns the sum of all numerical elements in the list.
- **`sorted(iterable, *, key=None, reverse=False)`** : Returns a new sorted list from the items in the iterable, without modifying the original list.
- **`list([iterable])`** : Creates a new list object. It can convert other iterables like tuples or strings into a list.
- **`enumerate(iterable, start=0)`** : Returns an enumerate object, which produces tuples of (index, value) when iterated over.
- **`reversed(sequence)`** : Returns a reverse iterator, which can then be converted back to a list using `list()`.

Nested list :

- A nested list in Python is a list that contains other lists as its elements, allowing for the creation of multi-dimensional data structures like matrices, tables, or hierarchical data.

Creating a Nested List:

- You can create a nested list using square brackets `[]`, just like a regular list, but with the inner lists as elements.

Example:

```
# A simple nested list representing student records: [Name, Age]
students = ["Harry", 20], ["Sarah", 22], ["Bruno", 21]]
print(students)
```

Output: [['Harry', 20], ['Sarah', 22], ['Bruno', 21]]

```
# A nested list representing a 3x3 matrix
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix)
```

Output: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Key Operations:

Accessing Elements in Nested List:

➤ To access elements, we use two indexes:

- First index → selects the row (inner list)
- Second index → selects the element inside that list

Example:

```
a = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
print(a[0][1])
print(a[2][2])
```

Output

20
90

Modifying Elements in Nested List:

- Elements inside nested lists can also be updated.

Example:

```
a = [[1, 2], [3, 4]]
```

```
a[1][0] = 10  
print(a)
```

Output

```
[[1, 2], [10, 4]]
```

Nested List Using Loop:

- We can print nested list elements using loops.

Example:

```
a = [[1, 2, 3], [4, 5, 6]]  
for i in a:  
    for j in i:  
        print(j)
```

Output

```
1  
2  
3  
4  
5  
6
```

Python Dictionary:

- A Python dictionary is a data structure that stores information in key-value pairs.
- While keys must be unique and immutable (like strings or numbers), the values can be of any data type, whether mutable or immutable.
- This makes dictionaries ideal for accessing data by a specific name rather than a numeric position like in list.

Example:

This example shows how a dictionary stores data using keys and values.

```
data = { "name": "Jake", "age": 22 }  
print(data)
```

Output

```
{'name': 'Jake', 'age': 22}
```

Creating a Dictionary:

- A dictionary is created by writing key-value pairs inside { }, where each key is connected to a value using colon (:).
- A dictionary can also be created using the [dict\(\)](#) function.

Example:

```
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}  
  
print(d1)  
  
# using dict() constructor  
  
d2 = dict(a = "Geeks", b = "for", c = "Geeks")  
  
print(d2)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}  
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

Operations of dictionaries:

1.Creating a Dictionary:

- A dictionary is created using curly braces {} with key–value pairs.

EX:

```
student = {"name": "Pooja", "age": 20, "course": "BCA"}
```

2. Accessing Elements:

- You can access values using keys.

Ex:

3. Adding Elements:

- You can add new key–value pairs.

Ex:

```
student["marks"] = 85
```

4. Updating Elements:

- Modify an existing value.

Ex:

```
student["age"] = 21
```

5.Looping Through Dictionary:

Ex:

```
for key in student:  
    print(key, student[key])
```

List of Python Dictionary Methods:

- Python provides several built-in methods for dictionaries that allow for efficient manipulation, access, and transformation of dictionary data.
- Here's a list of some important Python dictionary methods:

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>fromkeys()</u>	Returns a dictionary with the specified keys and value
<u>get()</u>	Returns the value of the specified key
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>setdefault()</u>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary

Lets Look at some Python dictionary methods with examples:

1. Dictionary clear() Method:

- The clear() method in Python is a built-in method that is used to remove all the elements (key-value pairs) from a dictionary.
- It essentially empties the dictionary, leaving it with no key-value pairs.

Example:

```
my_dict = {'1': 'Geeks', '2': 'For', '3': 'Geeks'}  
my_dict.clear()  
print(my_dict)
```

output:

```
{}
```

2. Dictionary get() Method:

- In Python, the get() method is a pre-built dictionary function that enables you to obtain the value linked to a particular key in a dictionary.
- It is a secure method to access dictionary values without causing a KeyError if the key isn't present.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
print(d.get('Name'))  
print(d.get('Gender'))
```

output:

```
Ram  
None
```

3. Dictionary items() Method:

- In Python, the items() method is a built-in dictionary function that retrieves a view object containing a list of tuples.
- Each tuple represents a key-value pair from the dictionary.
- This method is a convenient way to access both the keys and values of a dictionary simultaneously, and it is highly efficient.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
print(list(d.items())[1][0])  
print(list(d.items())[1][1])
```

output:

```
Age  
19
```

4. Dictionary keys() Method:

- The keys() method in Python returns a view object with dictionary keys, allowing efficient access and iteration.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
print(list(d.keys()))
```

output:

```
['Name', 'Age', 'Country']
```

5. Dictionary update() Method:

- Python's update() method is a built-in dictionary function that updates the key-value pairs of a dictionary using elements from another dictionary or an iterable of key-value pairs.
- With this method, you can include new data or merge it with existing dictionary entries.

Example:

```
d1 = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
d2 = {'Name': 'Neha', 'Age': '22'}  
d1.update(d2)  
print(d1)
```

output:

```
{'Name': 'Neha', 'Age': '22', 'Country': 'India'}
```

6. Dictionary values() Method:

- The values() method in Python returns a view object containing all dictionary values, which can be accessed and iterated through efficiently.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
print(list(d.values()))
```

output:

```
['Ram', '19', 'India']
```

7. Dictionary pop() Method:

- In Python, the pop() method is a pre-existing dictionary method that removes and retrieves the value linked with a given key from a dictionary.
- If the key is not present in the dictionary, you can set an optional default value to be returned.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
d.pop('Age')  
print(d)
```

output:

```
{'Name': 'Ram', 'Country': 'India'}
```

8. Dictionary popitem() Method:

- The popitem() method in Python dictionaries is used to remove and return the last inserted key-value pair as a tuple.
- If the dictionary is empty then it raises a KeyError.

Example:

```
d = {'Name': 'Ram', 'Age': '19', 'Country': 'India'}  
val = d.popitem()  
print(val)  
val = d.popitem()  
print(val)
```

output:

```
('Country', 'India')  
('Age', '19')
```

Populating a Dictionary:

- Populating means adding data (key–value pairs) to a dictionary.
- A dictionary stores data in the form:
key : value

Ways to Populate a Dictionary:

1. Creating with initial values:

- Creating with initial values means defining and filling the dictionary at the same time, instead of adding elements later.

Example:

```
student = {  
    "name": "Pooja",  
    "age": 20,  
    "course": "BCA"  
}  
print(student)
```

2. Adding elements one by one:

- Adding elements one by one means inserting key–value pairs individually into a dictionary after creating it.

Example:

```
student = {}  
student["name"] = "Pooja"  
student["age"] = 20  
student["course"] = "BCA"  
  
print(student)
```

3. Using update() method:

- The update() method is used to add multiple key–value pairs at once or modify existing values in a dictionary.

Example:

```
student = {"name": "Pooja"}  
student.update({"age": 20, "course": "BCA"})  
print(student)
```

4. Using loop (dynamic input):

- This method is used to add elements to a dictionary using user input at runtime (i.e., values are not fixed in the program).

Example:

```
student = {}  
n = 3  
for i in range(n):  
    key = input("Enter key: ")  
    value = input("Enter value: ")  
    student[key] = value  
  
print(student)
```

Traversing a Dictionary:

- Traversing means accessing or iterating through all elements of the dictionary.

Methods to Traverse:**1. Traversing Keys:**

```
student = {"name": "Pooja", "age": 20, "course": "BCA"}  
for key in student:  
    print(key)
```

Output:

```
name  
age  
course
```

2. Traversing Values:

```
for value in student.values():  
    print(value)
```

Output:

```
Pooja  
20  
BCA
```

3. Traversing Key–Value Pairs:

```
for key, value in student.items():  
    print(key, ":", value)
```

Output:

name : Pooja
age : 20
course : BCA

4. Using keys() explicitly:

```
for key in student.keys():  
    print(key, student[key])
```

Python Tuples:

- A tuple in Python is an immutable ordered collection of elements.
- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).
- Tuples can hold elements of different data types.
- The main characteristics of tuples are being ordered, heterogeneous and immutable.

Creating a Tuple:

- A tuple is created by placing all the items inside parentheses (), separated by commas.
- A tuple can have any number of items and they can be of different data types.

Example:

```
tup = ()  
print(tup)
```

```
# Using String  
tup = ('Geeks', 'For')  
print(tup)
```

```
# Using List  
li = [1, 2, 4, 5, 6]  
print(tuple(li))
```

```
# Using Built-in Function  
tup = tuple('Geeks')  
print(tup)
```

output:

```
()  
(('Geeks', 'For'))  
(1, 2, 4, 5, 6)  
(('G', 'e', 'e', 'k', 's'))
```

Creating a Tuple with Mixed Datatypes:

- Tuples can contain elements of various data types, including other tuples, [lists](#), [dictionaries](#) and even [functions](#).

Example:

```
tup = (5, 'Welcome', 7, 'Geeks')  
print(tup)  
  
# Creating a Tuple with nested tuples  
tup1 = (0, 1, 2, 3)  
tup2 = ('python', 'geek')  
tup3 = (tup1, tup2)  
print(tup3)  
  
# Creating a Tuple with repetition  
tup1 = ('Geeks',) * 3  
print(tup1)
```

output:

```
(5, 'Welcome', 7, 'Geeks')  
((0, 1, 2, 3), ('python', 'geek'))  
(('Geeks', 'Geeks', 'Geeks'))
```

Tuple Basic Operations:

Accessing of Tuples:

- We can access the elements of a tuple by using indexing and [slicing](#), similar to how we access elements in a list.

- Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple.
- Negative indexing starts from -1 for the last element and goes backward.

Example:

```
# Accessing Tuple with Indexing
tup = tuple("Geeks")
print(tup[0])
```

```
# Accessing a range of elements using slicing
print(tup[1:4])
print(tup[:3])
```

```
# Tuple unpacking
tup = ("Geeks", "For", "Geeks")
```

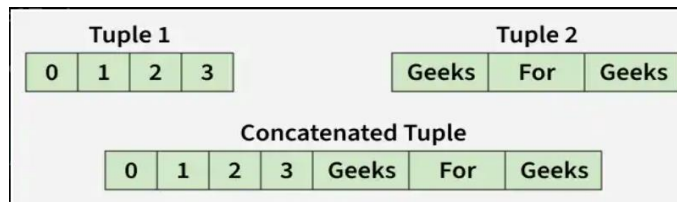
```
# This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
```

output:

```
G
('e', 'e', 'k')
('G', 'e', 'e')
Geeks
For
Geeks
```

Concatenation of Tuples:

- Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple.
- Note: Only the same datatypes can be combined with concatenation, an error arises if a list and a tuple are combined.



Example:

```
tup1 = (0, 1, 2, 3)
tup2 = ('Geeks', 'For', 'Geeks')
```

```
tup3 = tup1 + tup2
print(tup3)
```

output:

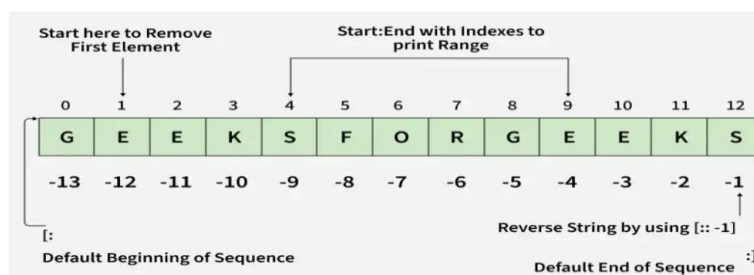
```
(0, 1, 2, 3, 'Geeks', 'For', 'Geeks')
```

Slicing of Tuple:

- [Slicing a tuple](#) means creating a new tuple from a subset of elements of the original tuple.
- The slicing syntax is :

`tuple[start:stop:step].`

Note: Negative Increment values can also be used to reverse the sequence of Tuples.



Example:

```
tup = tuple('GEEKSFORGEEKS')
```

```
# Removing First element  
print(tup[1:])
```

```
# Reversing the Tuple  
print(tup[::-1])
```

```
# Printing elements of a Range  
print(tup[4:9])
```

output:

```
('E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S')  
('S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G')  
('S', 'F', 'O', 'R', 'G')
```

Deleting a Tuple:

- Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using [del statement](#).

Note: Printing of Tuple after deletion results in an Error.

Example:

```
tup = (0, 1, 2, 3, 4)  
del tup
```

```
print(tup)
```

output:

```
ERROR!
```

```
Traceback (most recent call last):
```

```
File "<main.py>", line 6, in <module>
```

```
NameError: name 'tup' is not defined
```

Tuple Unpacking with Asterisk (*):

- In Python, the " * " operator can be used in tuple unpacking to grab multiple items into a list.
- This is useful when you want to extract just a few specific elements and collect the rest together.

Example:

```
tup = (1, 2, 3, 4, 5)
```

```
a, *b, c = tup
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

Output

```
1
```

```
[2, 3, 4]
```

```
5
```

- a gets the first item.
- c gets the last item.
- *b collects everything in between into a list.

Python Tuple built-in-methods:**1.Count() Method:**

- The count() method of Tuple returns the number of times the given element appears in the tuple.

Syntax: tuple.count(element)

Example:

```
# Creating tuples
```

```
Tuple1 = (0, 1, 2, 3, 2, 3, 1, 3, 2)
```

```
Tuple2 = ('python', 'geek', 'python',  
          'for', 'java', 'python')
```

```
# count the appearance of 3
```

```
res = Tuple1.count(3)
```

```
print('Count of 3 in Tuple1 is:', res)
```

```
# count the appearance of python
```

```
res = Tuple2.count('python')
```

```
print('Count of Python in Tuple2 is:', res)
```

output:

Count of 3 in Tuple1 is: 3

Count of Python in Tuple2 is: 3

2.Index() Method:

- The Index() method returns the first occurrence of the given element from the tuple.

Syntax:

```
tuple.index(element, start, end)
```

Parameters:

- element: The element to be searched.
- start (Optional): The starting index from where the searching is started
- end (Optional): The ending index till where the searching is done

Example : Using Tuple Index() Method

```
# Creating tuples
```

```
Tuple = (0, 1, 2, 3, 2, 3, 1, 3, 2)
```

```
# getting the index of 3
```

```
res = Tuple.index(3)
```

```
print('First occurrence of 3 is', res)
```

```
# getting the index of 3 after 4th
```

```
# index
```

```
res = Tuple.index(3, 4)
```

```
print('First occurrence of 3 after 4th index is:', res)
```

output:

First occurrence of 3 is 3

First occurrence of 3 after 4th index is: 5

Sets in Python:

- In Python, a set is an unordered collection of unique elements.
- Unlike lists or tuples, sets do not allow duplicate values i.e. each element in a set must be unique.
- Sets are mutable, meaning you can add or remove items after a set has been created.
- Sets are defined using curly braces **{}** or the built-in **set()** function.
- They are particularly useful for membership testing, removing duplicates from a sequence, and performing common mathematical set operations like union, intersection, and difference.

Creating a Set in Python:

- Creating a set in Python refers to defining and initializing a collection of unique elements.
- This includes specifying the elements that will be part of the set, ensuring that each element is unique within the set.
- You can create a set in Python using curly braces **{}** or the **set()** function :

Using Curly Braces:

- You can directly define a set by listing its elements within curly braces, separating each element by a comma as shown below –

Example:

```
my_set = {1, 2, 3, 4, 5}  
print (my_set)
```

It will produce the following result –
{1, 2, 3, 4, 5}

Using the set() Function:

- Alternatively, you can create a set using the **set()** function by passing an iterable (like a list or a tuple) containing the elements you want to include in the set

Example:

```
my_set = set([1, 2, 3, 4, 5])  
print (my_set)
```

We get the output as shown below –
{1, 2, 3, 4, 5}

Operations on sets:

1. **Adding elements to Sets:** [add\(\)](#) function is used to insert new elements into a set. It automatically ignores duplicates.

Example:

```
s = {"a", "b", "c"}  
s.add("d")  
print(s)
```

output:

```
{'c', 'd', 'a', 'b'}
```

2. **Union of Sets:** [union\(\)](#) function combines two sets and returns a new set with all unique elements.

Example:

```
a = {"x", "y"}  
b = {"y", "z"}  
u = a.union(b)  
print(u)
```

output:

```
{'z', 'y', 'x'}
```

3. **Intersection of Sets:** [intersection\(\)](#) function returns a new set containing elements that are common to both sets.

Example:

```
a = {1, 2, 3}  
b = {2, 3, 4}  
i = a.intersection(b)  
print(i)
```

output:

```
{2, 3}
```

4. **Difference of Sets:** [difference\(\)](#) function returns a set containing elements that are in the first set but not in the second.

Example:

```
a = {1, 2, 3}
b = {2, 3, 4}
d = a.difference(b)
print(d)
```

output:

```
{1}
```

5. **Clearing a Set:** `clear()` function removes all elements from a set, leaving it empty.

Example:

```
s = {1, 2, 3}
s.clear()
print(s)
```

output:

```
set()
```

python sets built-in-methods:

Functions Name	Description
<code>add()</code>	Adds a given element to a set
<code>clear()</code>	Removes all elements from the set
<code>copy()</code>	Returns a shallow copy of the set
<code>difference()</code>	Returns a set that is the difference between two sets
<code>difference_update()</code>	Updates the existing caller set with the difference between two sets
<code>discard()</code>	Removes the element from the set

Functions Name	Description
<u>frozenset()</u>	Return an immutable frozenset object
<u>intersection()</u>	Returns a set that has the intersection of all sets
<u>intersection_update()</u>	Updates the existing caller set with the intersection of sets
<u>isdisjoint()</u>	Checks whether the sets are disjoint or not
<u>issubset()</u>	Returns True if all elements of a set A are present in another set B
<u>issuperset()</u>	Returns True if all elements of a set A occupies set B
<u>pop()</u>	Returns and removes a random element from the set
<u>remove()</u>	Removes the element from the set
<u>symmetric_difference()</u>	Returns a set which is the symmetric difference between the two sets
<u>symmetric_difference_update()</u>	Updates the existing caller set with the symmetric difference of sets
<u>union()</u>	Returns a set that has the union of all sets
<u>update()</u>	Adds elements to the set

